

TSOpt guide 3: Designing for your programme

Contents

1	Targets, or no targets	2
2	Budgets with unit costs	3
3	Field reality: checks, lost seed and last year's data	3
4	Stratification, quotas and caps	4
5	Replicates	5
6	Goals: what are you trying to do?	5
7	Expert criteria and searches	6
8	Variance components from your own trials: <code>tso_variance()</code>	7
9	Hedging over models of gene action (kernels)	7
10	Breeding systems	8
10.1	Hybrids: which crosses to make and test	8
10.2	Clonal crops and dominance	9
10.3	Genebanks and diversity panels	9
10.4	Testcrosses: lines evaluated with testers	10
10.5	Three or more heterotic groups	10
11	Sparse multi-environment testing	10
12	Multi-trait index designs: which trait on which line	12
12.1	Robust to uncertain heritabilities and correlations	13
12.2	What is each trait worth? Proxy and high-throughput traits	13
12.3	Traits and environments together	14
13	Very large panels	14
14	Laying out the field: <code>tso_layout()</code>	15
15	Whom to genotype: <code>tso_genotype_set()</code>	16
16	Genotyping, imputation and plots under one budget: <code>tso_imputation_plan()</code>	17
17	Omics profiles: transcriptomes, metabolomes and spectra	18
18	From the plan to the field and the database	19
19	Training in one programme, predicting another: <code>tso_transfer()</code>	20

20 What is not here

20

A default design (see “TSOpt guide 2: Your first training set”) answers one question: which lines to phenotype if nothing else matters. A real programme has more to say. There are lines you must predict this season, a budget in money rather than plots, checks that go in every trial, seed that ran out, last year’s data, groups that must be represented, several sites and several traits. This guide shows how to tell `tso_design()` about each of these, in the order a breeder usually meets them. It then goes one step either side of the design: where each plot goes in the field (`tso_layout()`), whom to genotype in the first place (`tso_genotype_set()`), and whether another programme’s trials can train your predictions (`tso_transfer()`).

Everything runs on the example panel shipped with TSOpt: 300 inbred lines, 1,000 SNPs, four subpopulations and a cost per line.

```
library(TSOpt)
ex <- function(f) system.file("extdata", f, package = "TSOpt")
grp <- read.csv(ex("example_groups.csv")) # id, group
costs <- read.csv(ex("example_costs.csv")) # id, cost
tp <- tso_data(ex("example_panel.vcf.gz"),
              groups = setNames(grp$group, grp$id),
              cost = setNames(costs$cost, costs$id), verbose = FALSE)
ids <- tp$units$id
targets <- ids[251:300] # this season's new lines, to be predicted
```

1 Targets, or no targets

If you know which lines you will select among (this year’s new lines, the crosses you will make), give them as target. The design then chooses a training set that predicts *those* lines well, from the other candidates. Without target the design is *untargeted*: it makes the whole pool as predictable as possible, which is the right choice for a reference panel you will reuse for years.

```
p_all <- tso_design(tp, n = 60, verbose = FALSE)
p_tgt <- tso_design(tp, n = 60, target = targets, verbose = FALSE)
p_tgt
#> TSOpt plan · inbred · 60 lines of 250 candidates · 50 named targets
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8)
  ↳ Search greedy (robust CDmean: average over h2) (0.0 s)
#> Expected CDmean 0.229 (at h2 = 0.50) (exact, intercept projected: 0.229) vs
  ↳ random 0.089 ± 0.021 (better than 100% of 100 random sets)
#> Verdict GREEN optimisation pays off here: expected accuracy ~61.5% above random
#> Structure discrete groups (4 groups, silhouette 0.65) -> stratified allocation
  ↳ 12/12/11/25
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
  ↳ unreachable (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↳ tso_plot(plan, "divide")
#> Alleles 100.0% of target alleles present in training (0 markers not covered) ·
  ↳ 96.3% of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.15% to 8.07% of the CDmean (median 0.23%) · see
  ↳ tso_plot(plan, "value")
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

The printout is the plan in one screen. Expected gives the CDmean (the average reliability of the target predictions) against 100 random training sets of the same size; Verdict turns the difference into expected accuracy; Structure shows how the 60 lines were spread over the four groups; Coverage, Divide and Alleles flag targets that no training line can predict.

```
tso_plot(p_tgt, "pca")
```



What to look at: the targeted plan beats random sets by 62% in expected accuracy, the untargeted one by only 5%. Knowing your targets is the single most valuable thing you can tell TSOpt. None of the targets is in the training set: TRUE.

2 Budgets with unit costs

Plots do not all cost the same: seed multiplication, a remote site, an extra replicate. Give the costs to `tso_data(cost = )` (done above) and a budget instead of `n`. Units are then chosen by information gained per unit of cost.

```
p_bud <- tso_design(tp, budget = 150, target = targets, verbose = FALSE)
c(lines = nrow(p_bud$selected), spent = p_bud$spent,
  mean_cost_chosen = round(mean(p_bud$selected$cost), 2),
  mean_cost_panel = round(mean(tp$units$cost), 2))
#>      lines      spent mean_cost_chosen mean_cost_panel
#>      95.00     149.26          1.57          2.00
```

A budget of 150 bought 95 lines. Their average cost, 1.57, is below the panel average of 2: cheap lines are preferred when they carry similar information, but an expensive line that is the only close relative of several targets is still bought.

3 Field reality: checks, lost seed and last year's data

Three arguments cover what the field team will tell you:

- `must_include`: checks, parents or lines already sown. They are forced in and count towards `n` and budget.
- `exclude`: no seed, lost, off-type. These can never be chosen.

- history: lines phenotyped in earlier seasons. They inform the design for free and do not count towards n. history_weight says how much an old record is worth relative to a new plot (1 = as good; 0.5 for an older or less related season).

```
p_real <- tso_design(tp, n = 60, target = targets,
                    must_include = ids[1:3], exclude = ids[4:10],
                    history = ids[11:40], history_weight = 0.8,
                    verbose = FALSE)
head(p_real$selected[, c("order", "id", "group", "forced")], 5)
#>   order  id group forced
#> 1     1  L001   G1   TRUE
#> 2     2  L002   G1   TRUE
#> 3     3  L003   G1   TRUE
#> 4     4  L234   G4  FALSE
#> 5     5  L226   G4  FALSE
```

The three checks come first, marked forced. The 30 lines from last year already cover part of genetic space, so the 57 free plots move elsewhere: 10 of the 60 lines differ from the plan without these constraints. The expected CDmean rises from 0.229 to 0.235 because the history counts too.

What to look at: always pass last season's phenotyped lines as history. It is free information, and without it the design wastes plots on lines that are already well predicted.

4 Stratification, quotas and caps

When the panel has discrete groups (subpopulations, heterotic groups, programmes), stratify = "auto" (the default) spreads the set over them in proportion to what the targets need. "groups" always stratifies by the groups you gave, and "none" lets the criterion decide alone. If management has already agreed numbers, give them as group_quota.

```
p_q <- tso_design(tp, n = 60, verbose = FALSE,
                 group_quota = c(G1 = 20, G2 = 20, G3 = 10, G4 = 10))
rbind(auto = table(p_all$selected$group), quota = table(p_q$selected$group))
#>      G1 G2 G3 G4
#> auto  15 15 15 15
#> quota 20 20 10 10
```

Caps stop the set from piling up in one place: max_per_family (lines per family), max_per_parent (hybrids per parent, see below) and max_per_env (units per environment). The example files have no family column, so for illustration we cut the panel into 30 clusters of close relatives and treat each as a family.

```
K <- tp$K_lines
fam <- cutree(hclust(as.dist(max(K) - K), "average"), 30)
fam <- setNames(paste0("F", fam), rownames(K))
tpf <- tso_data(ex("example_panel.vcf.gz"), families = fam, verbose = FALSE)
p_free <- tso_design(tpf, n = 60, verbose = FALSE)
p_cap <- tso_design(tpf, n = 60, max_per_family = 3, verbose = FALSE)
c(no_cap = max(table(p_free$selected$family)),
  cap_3 = max(table(p_cap$selected$family)))
#> no_cap cap_3
#>      4      3
```

The cap costs little: CDmean 0.346 with it, 0.346 without.

5 Replicates

reps is the maximum number of plots per unit. With `reps = 2`, the engine may give a line a second plot when that is more informative than a new line, which happens at low heritability, when a second plot of a well-connected line reduces noise more than a first plot of a line nobody is related to. `n` then counts plots, and `selected` has one row per plot, with the number of plots of that line in `plots`.

```
p_rep <- tso_design(tp, n = 80, reps = 2, h2 = 0.1, stratify = "none",
                  verbose = FALSE)
table(plots_per_line = p_rep$selected$plots[!duplicated(p_rep$selected$id)])
#> plots_per_line
#> 1 2
#> 76 2
```

At $h^2 = 0.1$, 2 lines get two plots and 78 distinct lines are tested. At moderate heritability the engine almost never replicates: new lines are worth more.

6 Goals: what are you trying to do?

goal says, in breeder language, what the training set is for. Each goal uses a different criterion:

goal	criterion	use when
"accuracy" (default)	CDmean	general use
"gain"	expected mean of the top select_top fraction	strong truncation selection
"families"	CDpop: contrasts between family means	targets are crosses, not lines
"information"	D-optimality, no target	no target set is known
"reference_panel"	untargeted CDmean, stratified	reusable panels, genebanks
"correlation"	r-score (Ou and Liao 2019)	direct expected correlation

To compare them fairly we score every set on the same yardstick: CDmean on the 50 targets at $h^2 = 0.5$, and the mean relationship inside the set (lower = more diverse). `information` and `reference_panel` do not use targets, so they are designed without them, but from the same candidates (every line that is not a target).

```
goals <- c("accuracy", "gain", "correlation", "information", "reference_panel")
tab <- do.call(rbind, lapply(goals, function(gl) {
  tg <- if (gl %in% c("information", "reference_panel")) NULL else targets
  p <- tso_design(tp, n = 60, target = tg, goal = gl, select_top = 0.1,
                 candidates = setdiff(ids, targets), verbose = FALSE)
  s <- p$selected$id
  data.frame(goal = gl, criterion = p$value$criterion,
             cdmean_targets = tso_score(tp, s, target = targets)[["cdmean"]],
             mean_kinship = mean(K[s, s][upper.tri(K[s, s])]))
}))
knitr::kable(tab, digits = 3)
```

goal	criterion	cdmean_targets	mean_kinship
accuracy	cdmean	0.229	0.001

goal	criterion	cdmean_targets	mean_kinship
gain	gain	0.227	0.000
correlation	rscore	0.229	0.001
information	dopt	0.084	-0.003
reference_panel	cdmean	0.112	-0.006

What to look at: the three goals that know the targets (accuracy, gain, correlation) all reach a CDmean of about 0.229 on them: which of the three you use matters little. The two untargeted goals reach only 0.084 and 0.112, close to random sampling (0.089). An untargeted panel is a good general-purpose panel, but it is not tuned to this season's lines. Pick the goal that matches your decision, and give the targets whenever you know them.

7 Expert criteria and searches

`tso_criteria()` lists all 18 criteria with the search each one uses. Two expert choices are worth knowing.

Worst case instead of average. `criterion = "cdmin"` maximises the reliability of the *least* reliable target, so no target is left unpredictable (pre-breeding, genebanks, very diverse targets).

A harder search. The default greedy search with an exchange pass is close to the optimum on most panels. `search = "tabu"` (or "ga") searches longer, and restarts adds multi-start runs from perturbed starting sets (spread over cores on Linux and macOS). Use them when a few extra hundredths of CDmean justify a longer run.

```
p_min <- tso_design(tp, n = 60, target = targets, criterion = "cdmin",
  verbose = FALSE)
p_tabu <- tso_design(tp, n = 60, target = targets, h2 = 0.5, search = "tabu",
  restarts = 3, verbose = FALSE)
round(c(greedy = p_tgt$value$cdmean, cdmin = p_min$value$cdmean,
  tabu_3_restarts = p_tabu$value$cdmean), 4)
#>      greedy      cdmin tabu_3_restarts
#>    0.2291      0.2276      0.2291
```

Your own criterion. Any function `f(selected_ids, data)` that returns one number can be the criterion. It is maximised unless you say `direction = "minimize"`. Here we minimise the mean relationship inside the set, for a maximally diverse panel.

```
similarity <- function(ids, data) mean(data$K[ids, ids])
p_div <- tso_design(tp, n = 30, criterion = similarity, direction = "minimize",
  iterations = 200, verbose = FALSE)
set.seed(1)
c(diverse_set = similarity(p_div$selected$id, tp),
  random_set = similarity(sample(ids, 30), tp))
#> diverse_set random_set
#> 0.01376490 0.03278938
```

Forgetting direction is a classic mistake: the search then *maximises* similarity and returns a clump of near-duplicates. TSOpt checks for this and warns:

```
w <- tryCatch(tso_design(tp, n = 30, criterion = similarity, iterations = 200,
  verbose = FALSE),
  warning = function(w) conditionMessage(w))
cat(strwrap(w, 80), sep = "\n")
```

```
#> Your criterion scores a set of near-duplicate relatives higher than every
#> random set. Criteria that measure error, PEV or similarity must be minimised:
#> if that is the case, set direction = "minimize".
```

8 Variance components from your own trials: `tso_variance()`

Heritabilities, genetic correlations and environment covariances are usually guessed. `tso_variance()` estimates them by REML (the average-information algorithm) from your historical trials, with standard errors, and `tso_data(variance = )` uses the estimates. Robust designs then average over scenarios drawn from their sampling distribution instead of fixed grids.

```
ph <- read.csv(ex("example_pheno.csv"))           # id, y
v <- tso_variance(tp, ph)
v
#> TSOpt variance components (AI-REML) · single · 300 records on 300 lines · logL
  ↪ -259.84 · converged in 4 iterations
#> heritability: y 0.42 (SE 0.10)
#> 5 scenarios drawn from the sampling distribution. Next: tso_data(geno, variance =
  ↪ v)
tp_v <- tso_data(ex("example_panel.vcf.gz"), variance = v, verbose = FALSE)
tp_v$h2_grid      # replaces h2 = 0.2, 0.5, 0.8 in robust designs
#> [1] 0.270 0.422 0.499
```

With several traits (`traits = c("yield", "protein")`, one row per line) it estimates the genetic and residual covariance between traits; with `env = "site"` (one row per line and site) the genetic covariance between sites and a residual variance per site; with `kernels =` the share of each kernel.

9 Hedging over models of gene action (kernels)

Robust CDmean hedges over the heritability you do not know (guide 2). The relationship matrix is a second assumption: the additive matrix says that genetic value is a sum of allele effects. `kernels =` averages the criterion over several models of gene action instead: "additive" (VanRaden), "dominance" (Vitezica, for diploids) and "gaussian", the RKHS kernel, which captures epistasis between loci. The set is then not tuned to one model that may be wrong.

```
kn <- c("additive", "dominance", "gaussian")
p_add <- tso_design(tp, n = 60, target = targets, h2 = 0.5, verbose = FALSE)
p_ker <- tso_design(tp, n = 60, target = targets, h2 = 0.5, kernels = kn,
  verbose = FALSE)
# the additive plan scored under the same three kernels: force all its lines in
p_chk <- tso_design(tp, n = 60, target = targets, h2 = 0.5, kernels = kn,
  must_include = p_add$selected$id, verbose = FALSE)
round(rbind(additive_only = p_chk$value$cd_by_scenario,
  hedged = p_ker$value$cd_by_scenario), 4)
#> additive dominance gaussian
#> additive_only 0.2291 0.1612 0.1005
#> hedged 0.2284 0.1629 0.1006
```

`p_ker$value$cd_by_scenario` holds the CDmean under each kernel (and, without a fixed `h2`, under each kernel x heritability pair; the plan prints them in its Scenarios line). The kernels are not on a common scale, so compare the rows within a column, not the columns with each

other.

What to look at: the two plans share 44 of their 60 lines, and under every kernel their CDmeans differ by at most 0.0017. On this panel the additive plan was already robust to the model of gene action, so the hedge costs nothing and changes little. It matters most for clones, hybrids and traits with strong epistasis, where the kernels disagree about who is related to whom. kernels works for line-level data with marker genotypes (not hybrids, environments or traits).

10 Breeding systems

One engine serves every breeding system. What changes is the *unit* you can phenotype (a line, a hybrid, a clone, a line in one environment) and how units are related.

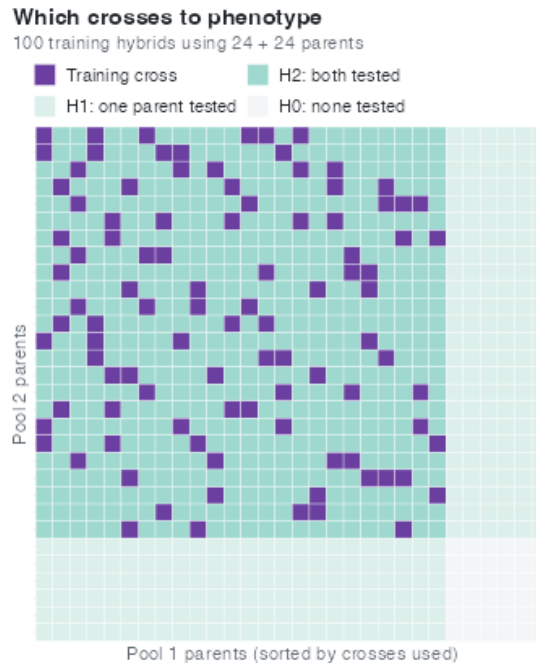
10.1 Hybrids: which crosses to make and test

A hybrid's genetic value is GCA of the female + GCA of the male + SCA. Give the two parent pools with pools, the share of SCA with sca_share, and the parents that can be used in training crosses with crossable (new parents with no seed yet can be targets, but not training parents).

```
hp <- tso_simulate_panel(n = 60, markers = 800, groups = 2, fst = 0.3, seed = 2)
pid <- rownames(hp$dosage)
th <- tso_data(hp, system = "hybrid", pools = list(pid[1:30], pid[31:60]),
              crossable = c(pid[1:24], pid[31:54]), sca_share = 0.2,
              verbose = FALSE)
ph <- tso_design(th, n = 100, max_per_parent = 6, verbose = FALSE)
ph
#> TSOpt plan · hybrid · 100 hybrids of 576 candidates · untargeted (whole pool)
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8)
#> ↳ Search greedy (robust CDmean: average over h2) (0.6 s)
#> Expected CDmean 0.467 (at h2 = 0.50) (exact, intercept projected: 0.467) vs
#> ↳ random 0.440 ± 0.004 (better than 100% of 100 random sets)
#> Verdict AMBER modest gain (~3.0% accuracy over random); worth using, but do not
#> ↳ expect much
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
#> ↳ unreachable (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
#> ↳ tso_plot(plan, "divide")
#> Plot value dropping one plot costs 0.37% to 0.51% of the CDmean (median 0.44%) · see
#> ↳ tso_plot(plan, "value")
#> Classes untested hybrids: H0 = 36, H1 = 288, H2 = 476
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

The 30 x 30 pools give 900 possible hybrids, of which 576 involve only crossable parents. max_per_parent = 6 stops the design from leaning on a few favourite parents. The Classes line splits the untested hybrids by how many of their parents are in training: H2 (both), H1 (one) and H0 (neither).

```
tso_plot(ph, "hybrid_grid")
```



predict tells the design which class matters to you. "H2" for filling in a factorial of known parents, "H1" when new lines will be crossed to established testers, "H0" for crosses between new parents on both sides.

```
sapply(c(all = "all", H2 = "H2", H1 = "H1", H0 = "H0"), function(cl) {
  p <- tso_design(th, n = 100, max_per_parent = 6, predict = cl,
    verbose = FALSE)
  round(p$value$cdmean, 3)
})
#> all      H2      H1      H0
#> 0.467 0.538 0.367 0.202
```

What to look at: the reliability you can hope for falls steeply from H2 to H0. Predicting crosses between two new parents is hard whatever you phenotype.

10.2 Clonal crops and dominance

For clones (potato, cassava, banana, sugarcane) use `system = "clonal"`. For diploids a dominance kernel is added (`dom_share`, default 0.2 of the genetic variance), because a clone passes on its whole genotype, not just the additive part. Polyploid dosages (0 to ploidy) are read directly and the ploidy is detected from the file.

```
op <- tso_simulate_panel(n = 150, markers = 600, type = "outbred", seed = 6)
tc <- tso_data(op, system = "clonal", verbose = FALSE)
tail(tc$log, 2)
#> [1] "Kinship additive (VanRaden, base frequencies), mean diagonal scaled to 1"
#> [2] "Kinship additive + dominance (dominance share 0.2)"
```

10.3 Genebanks and diversity panels

`system = "genebank"` makes the untargeted reference panel the natural design: `goal = "reference_panel"` stratifies whenever the collection has groups, because in a genebank structure

carries real signal.

```
gb <- tso_simulate_panel(n = 300, markers = 800, groups = 5, fst = 0.25,
                        seed = 4)
tg <- tso_data(gb, system = "genebank", verbose = FALSE)
pg <- tso_design(tg, n = 60, goal = "reference_panel", verbose = FALSE)
table(pg$selected$group)
#>
#> G1 G2 G3 G4 G5
#> 12 12 12 12 12
```

10.4 Testcrosses: lines evaluated with testers

In hybrid programmes a new line is evaluated as a testcross with one or a few testers from the other heterotic group. `system = "testcross"` makes a unit one line x tester testcross, with genetic value $GCA(\text{line}) + SCA(\text{line} \times \text{tester})$. Testers behave like environments: each has its own mean, lines can be spread over them (`max_per_line = 1` for sparse testcrossing), and the design predicts the lines' **GCA** by default.

```
tc <- tso_data(ex("example_panel.vcf.gz"), system = "testcross", testers = ids[1:3],
              verbose = FALSE)
p_tc <- tso_design(tc, n = 60, max_per_line = 1, verbose = FALSE)
table(p_tc$selected$tester) # testcrosses per tester
#>
#> L001 L002 L003
#> 16 24 20
p_tc$spec$value
#> [1] "gca"
```

Testers that are genotyped get their genomic relationships for the SCA part; otherwise they are taken as unrelated.

10.5 Three or more heterotic groups

With `pools` a named list of three or more groups, crosses are made between the groups you list in `cross_groups` (default: every pair). The single cross $i \times j$ has genetic value $a_i + a_j + s_{ij}$, so crosses sharing a parent, or with related parents, are related: $\text{Cov}(g_{ij}, g_{kl}) = \sigma_a(K_{ik} + K_{il} + K_{jk} + K_{jl}) + \sigma_s(K_{ik}K_{jl} + K_{il}K_{jk})$.

```
h3 <- tso_data(ex("example_panel.vcf.gz"), system = "hybrid", verbose = FALSE,
              pools = list(A = ids[1:10], B = ids[76:85], C = ids[151:160]),
              cross_groups = list(c("A", "B"), c("A", "C")))
table(paste(h3$units$group1, "x", h3$units$group2))
#>
#> A x B A x C
#> 100 100
```

11 Sparse multi-environment testing

With `envs`, a unit is a line in one environment. The design decides which line goes to which site, and borrows information between sites through their genetic correlation. Give `env_cor` as one number, a correlation matrix, or a genetic *covariance* matrix estimated from earlier trials (site-specific genetic variances on the diagonal). `env_noise` gives the relative residual variance of each site: a noisy site (2) is worth half a plot at a clean one (1).

```

envs <- c("E1", "E2", "E3")
Om <- matrix(c(1, .7, .4,
               .7, 1, .5,
               .4, .5, 1), 3, dimnames = list(envs, envs))
tm <- tso_data(ex("example_panel.vcf.gz"), envs = envs, env_cor = Om,
               env_noise = c(E1 = 1, E2 = 2, E3 = 1), verbose = FALSE)

```

`max_per_line = 1` tests every line in one environment only (fully sparse testing), and `min_per_env` keeps every site in the trial.

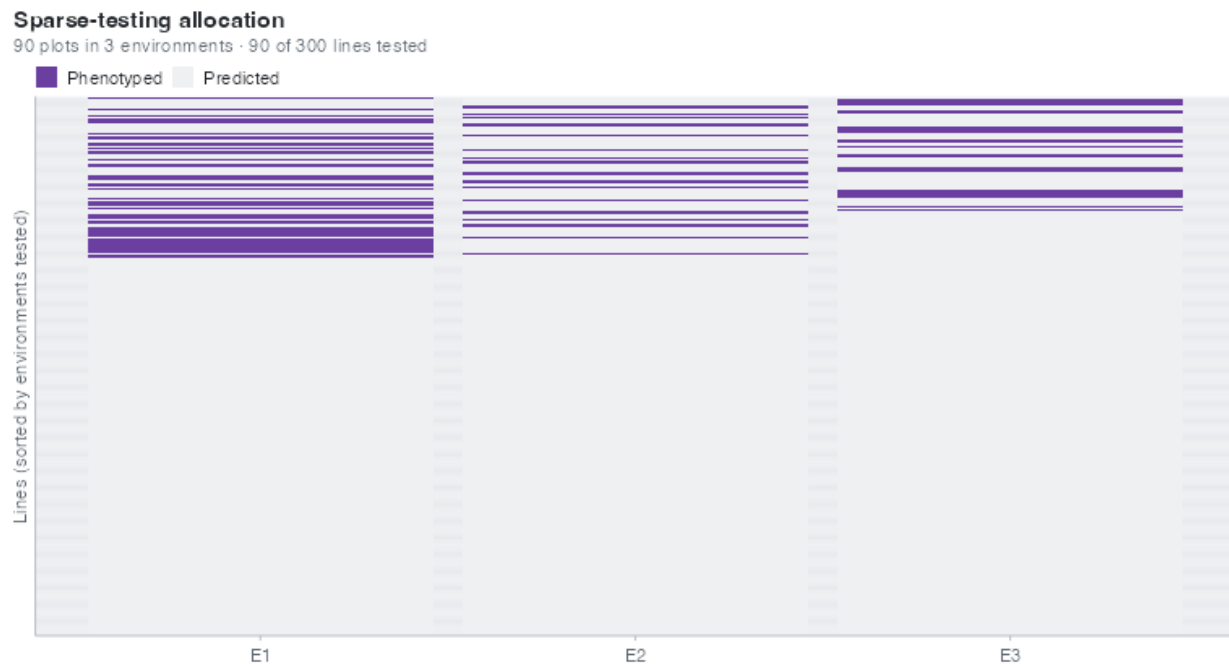
```

pm <- tso_design(tm, n = 90, target = targets, max_per_line = 1,
                 min_per_env = 20, verbose = FALSE)
pm$envs$allocation
#>
#> E1 E2 E3
#> 46 22 22
pm$spec$fixed
#> [1] "intercept + 2 fixed column(s) (exact)"

```

The noisy site E2 gets only the minimum. E1 is clean and closely correlated with E2 (0.7), so a plot in E1 also tells you much about E2: it takes the most plots. The environment means are added as fixed effects automatically (`pm$spec$fixed` above): the design is scored on the genetic values only, not on differences between sites.

```
tso_plot(pm, "met_allocation")
```



`env_target` says what the design should predict well:

- "mean" (default): each line's genetic value averaged over the environments;
- "cells": every line x environment cell;
- the name of one environment, for example a new site whose correlation with the others is in `env_cor`.

```

sapply(c("mean", "cells", "E3"), function(et) {
  p <- tso_design(tm, n = 90, target = targets, max_per_line = 1,

```

```

      min_per_env = 20, env_target = et, verbose = FALSE)
table(factor(p$selected$env, levels = envs))
})
#>      mean cells E3
#> E1    46    43 20
#> E2    22    20 20
#> E3    22    27 50

```

What to look at: when one site is the target (E3), most plots move to it; when the target is the mean, the plots follow the correlations and the noise.

12 Multi-trait index designs: which trait on which line

Breeders select on an index, and traits differ in cost. Yield needs a plot and a harvest, protein a lab assay, NDVI almost nothing once the plot exists. With traits, a unit is *one trait measured on one line*: give the heritabilities (trait_h2), the genetic correlations (gcor), the cost of each measurement (trait_cost) and the cost of growing a line at all (plot_cost, paid once per line). With index =, the design buys the cells that most improve the reliability of the index.

```

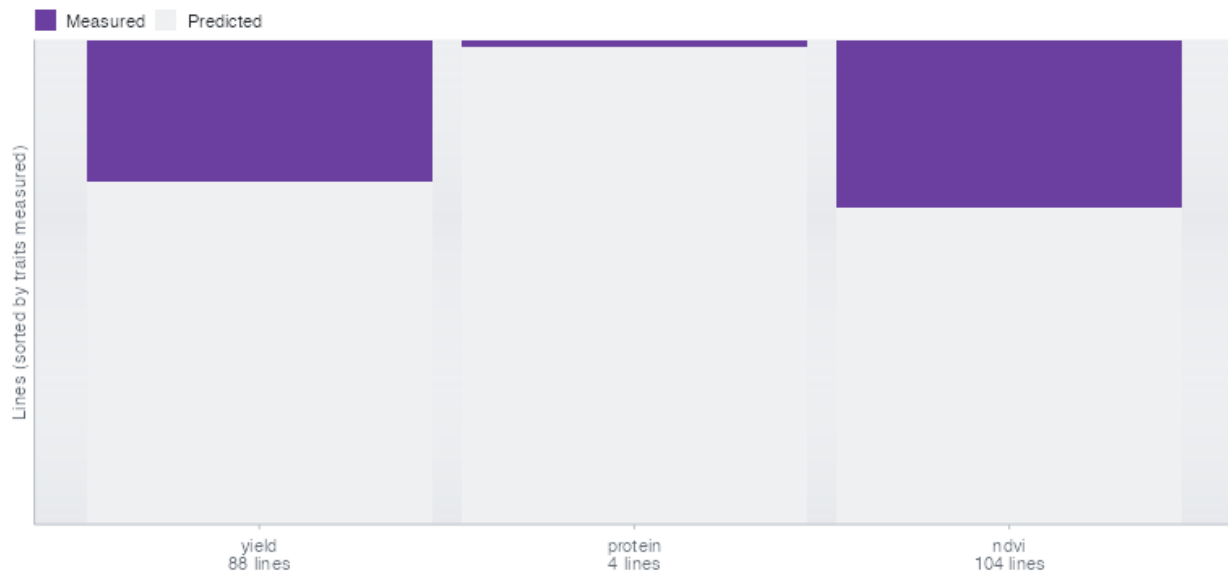
tr <- c("yield", "protein", "ndvi")
R <- matrix(c(1, 0.5, 0.6,
              0.5, 1, 0.3,
              0.6, 0.3, 1), 3, dimnames = list(tr, tr))
tt <- tso_data(ex("example_panel.vcf.gz"), traits = tr,
              trait_h2 = c(0.3, 0.5, 0.6), gcor = R,
              trait_cost = c(20, 35, 0.5), plot_cost = 10, verbose = FALSE)
pt <- tso_design(tt, budget = 3000, index = c(yield = 1, protein = 0.4),
              verbose = FALSE)
pt$traits
#>      trait lines weight h2
#> 1  yield    88    1.0 0.3
#> 2 protein     4    0.4 0.5
#> 3  ndvi   104    0.0 0.6

```

NDVI has index weight 0, yet it is measured on 104 lines: it is cheap and correlated with yield, so it is cheap information about the index. Protein is expensive and gets only 4 lines. That is a very different trial from “grow some lines and measure everything on them”. max_per_line also works here, as the maximum number of traits per line.

```
tso_plot(pt, "trait_allocation")
```

Which trait to measure on which line
104 lines grown · index yield 1, protein 0.4, ndvi 0



Each trait gets its own mean in the model, estimated exactly as a fixed effect, so the design is scored on the genetic values and not on differences between trait scales. Every trait that is measured at all gets at least two units, because the first measurement of a trait only estimates its mean.

12.1 Robust to uncertain heritabilities and correlations

Trait heritabilities and genetic correlations are usually estimated from a few small trials, and are noisy. By default (`trait_uncertainty = "auto"`) the design averages the index criterion over five scenarios: the values as given, heritabilities 40% lower and 40% higher, and correlations halved towards 0 or moved halfway towards ± 1 . This is the multi-trait counterpart of robust CDmean. The plan prints the CDmean under each scenario:

```
out <- capture.output(print(pt))
cat(grep("^Scenarios", out, value = TRUE), sep = "\n")
#> Scenarios  CDmean given 0.345 · h2_lower 0.257 · h2_higher 0.416 · gcor_weaker 0.292
↪      · gcor_stronger 0.423
```

What to look at: the same set ranges from a CDmean of 0.257 if the heritabilities are lower than you think to 0.423 if the correlations are stronger. The spread tells you how much your trait parameters matter; the design is chosen to do well across all five. Use `trait_uncertainty = "none"` only when the parameters come from large, reliable trials.

12.2 What is each trait worth? Proxy and high-throughput traits

`tso_trait_value()` puts a price on each trait of a multi-trait plan: what the design spent on it, the information one more measurement adds per unit of cost, how many measurements of it carry the information of one measurement of the main trait (the **exchange rate**), and how much index CDmean is lost when the trait is dropped and its money spent on the others.

```
tv <- tso_trait_value(pt)
tv
#> TSOpt trait value · 3 traits · index CDmean of the plan 0.345 · exchange rates in
↪      yield measurements
```

```
#>   trait weight cost lines budget value per cost worth 1 ref
#>   yield   1.0 20.0   88   90%           1.00         1.0
#>   protein  0.4 35.0    4    7%           0.94         0.7
#>   ndvi     0.0  0.5  104    3%           0.91         2.6
#>   CDmean lost without it
#>                   15.8%
#>                   0.5%
#>                   13.4%
#> -> yield gives the most information per unit of cost at the margin; ndvi (index
↪   weight 0) is measured as a proxy.
```

What to look at: NDVI has index weight 0 and takes 2.7% of the budget, yet dropping it loses 13.4% of the index CDmean. The exchange rate says 2.6 NDVI scans carry the information of one yield plot, at a fraction of the cost. This is the case for drone, NIRS and other high-throughput proxies, measured in the currency of your own programme. A trait whose loss is close to 0 (here protein, 0.5%) can be cut from this trial with little harm.

12.3 Traits and environments together

Give traits and envs together and a unit is one trait on one line in one environment. The genetic covariance combines the three (environments x traits x lines), there is one exact mean per trait x environment, and plot_cost is paid once for each line grown in each environment, however many traits are measured on it.

```
tte <- tso_data(ex("example_panel.vcf.gz"), traits = c("yield", "ndvi"),
               trait_h2 = c(0.3, 0.6), gcor = R[c(1, 3), c(1, 3)],
               trait_cost = c(yield = 20, ndvi = 0.5), plot_cost = 10,
               envs = c("E1", "E2"), env_cor = 0.6, verbose = FALSE)
pte <- tso_design(tte, budget = 3000, index = c(yield = 1, ndvi = 0),
                 target = targets, verbose = FALSE)
table(trait = pte$selected$trait, env = pte$selected$env)
#>      env
#> trait  E1 E2
#>   ndvi  85 76
#>   yield 34 31
pte$spec$fixed
#> [1] "intercept + 3 fixed column(s) (exact)"
```

What to look at: the design chose 136 lines and split them over both sites, with 161 NDVI and 65 yield measurements. The fixed effects are the 4 trait x environment means (an intercept plus 3 columns). env_target works as for a single trait.

13 Very large panels

tso_design() works on the full N x N relationship matrix, which is out of reach beyond about 15,000 lines. tso_design_large() gives the same robust CDmean design by working with rank genetic components taken straight from the markers, so it never builds an N x N matrix. We use a simulated panel of 3,000 lines so this guide knits quickly; the same call works for 100,000.

```
big <- tso_simulate_panel(n = 3000, markers = 1500, seed = 5)
p_big <- tso_design_large(big, n = 200, rank = 200, verbose = FALSE)
p_big
#> TSOpt plan (large panel) · 200 lines of 3,000 candidates · 3,000 lines x 1,435
↪   markers · untargeted (whole pool)
```

```
#> Features    randomised SVD (1 power iteration), rank 200, 78.5% of trace(K) captured
  ↳ (5 MB, no N x N matrix)
#> Criterion    CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) on
  ↳ 3,000 contrasts    Search greedy (robust CDmean: average over h2) (2.9 s)
#> Expected    CDmean 0.472 (at h2 = 0.50) (exact, intercept projected: 0.472) vs
  ↳ random 0.437 ± 0.001 (better than 100% of 20 random sets)
#> Verdict      AMBER modest gain (~3.9% accuracy over random); worth using, but do not
  ↳ expect much
#> Coverage    nearest training relative >= 0.20 for 100% of targets · 0 targets
  ↳ unreachable (< 0.05)
#> Next        plot(plan) · write.csv(plan$selected, "trial.csv")
```

What to look at: the Features line. The 200 components capture 78.5% of the genetic variance (trace of K). Below about 70%, increase rank. `tso_design_large()` takes target, must_include, exclude and candidates like `tso_design()`, but not quotas, caps, environments or traits.

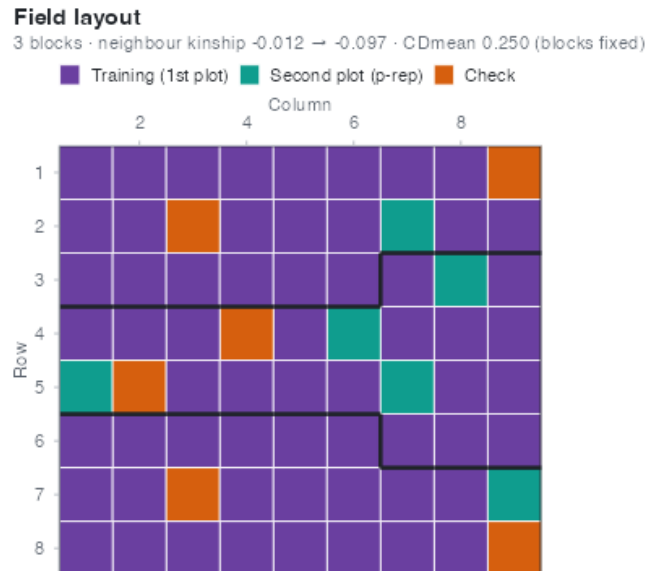
Large multi-environment and multi-trait designs. With traits or environments the unit covariance is a Kronecker product ($\Omega_E \otimes G_T \otimes K$), and above 8,000 line x trait x environment cells `tso_data()` keeps it in that factorised form (`implicit = TRUE`). Designs are identical to the dense form (checked to 1e-10 in the tests), and 1,000 lines x 15 sites (15,000 units, whose dense matrix would take 1.7 GB) are designed in about two and a half minutes. Criteria from the CD/PEV/gain family and the greedy, exchange and tabu searches work on it.

14 Laying out the field: `tso_layout()`

`tso_design()` decides *which* lines to phenotype; `tso_layout()` decides *where* each plot goes on a rows x cols grid. It deals the lines to blocks so that every block gets the same mix of genetic groups (block effects are then not confounded with groups), spreads the checks through every block (check_reps plots per check per block), gives a second plot to a fraction prep of the lines (a partially replicated design), and then moves lines around by simulated annealing so that close relatives are not planted next to each other. Plots are numbered in serpentine order, and each block is a band of consecutive rows.

```
lay <- tso_layout(p_tgt, rows = 8, cols = 9, blocks = 3,
  checks = c("CHK1", "CHK2"), prep = 0.1, seed = 1)
lay
#> TSOpt field layout · 8 x 9 plots · 3 blocks (serpentine runs (row bands))
#> Plots      60 lines + 6 second plots (p-rep) + 6 check plots (2 checks) · 72 of 72
  ↳ plots used
#> Blocks      24/24/24 entries per block · balanced over 4 strata (plan strata)
#> Neighbours mean kinship of neighbouring plots -0.012 -> -0.097 after 20,000
  ↳ annealing steps; close relatives (>= 0.5) adjacent: 1 -> 0
#> Expected    CDmean 0.250 with block effects fixed · 0.253 ignoring blocks (cost of
  ↳ blocking 0.0029)
#> p-rep      second plots chosen by plan$selected$value
#> Next        plot(layout) · as.data.frame(layout) for the field book
head(as.data.frame(lay), 4)
#>   plot row col block  id      type rep stratum
#> 1    1  1  1    1    1 L249 training  1      G4
#> 2    2  2  1    2    1 L193 training  1      G3
#> 3    3  3  1    3    1 L250 training  1      G4
#> 4    4  4  1    4    1 L003 training  1      G1
```

```
tso_plot(lay)
```



The second plots go to the lines with the highest *value* in the plan (the CDmean lost if the line were dropped, see guide 5), always in a different block from the first plot. `as.data.frame(lay)` is the field book: plot, row, column, block, ID, type and replicate.

What to look at: the Neighbours and Expected lines. The search lowered the mean kinship between neighbouring plots from -0.012 (random placement) to -0.097, and the pairs of close relatives (≥ 0.5) planted side by side from 1 to 0. Fitting the blocks costs 0.0029 of CDmean (0.250 with block effects fixed, 0.253 without): the price of the protection against field trends. Lay out each site of a multi-environment plan with its own call.

15 Whom to genotype: `tso_genotype_set()`

One step before phenotyping comes genotyping. With money to genotype only n individuals at high density (an imputation reference panel, or a genotyping budget), `tso_genotype_set()` chooses them from what is known before genotyping: a pedigree (`tso_data(pedigree = )`), a cheap low-density relationship matrix (`tso_data(kinship = )`) or low-density genotypes. The default method, "coverage", maximises the average relationship between each individual and its closest genotyped relative, which finds the key ancestors that tie the pedigree together.

We simulate a pedigree of 10 founders and three generations of 30 random matings:

```
set.seed(3)
ped <- data.frame(id = paste0("F", 1:10), p1 = NA, p2 = NA, gen = 0)
for (g in 1:3) {
  par <- ped$id[ped$gen == g - 1]
  m <- t(replicate(30, sample(par, 2)))
  ped <- rbind(ped, data.frame(id = paste0("G", g, "_", 1:30), p1 = m[, 1],
                               p2 = m[, 2], gen = g))
}
gs <- tso_genotype_set(tso_data(pedigree = ped[, 1:3], verbose = FALSE), n = 10)
gs
#> TSOpt genotyping set · 10 of 100 candidates · all 100 individuals · relationships
  ↳ from the pedigree
#> Method coverage (facility location) (0.0 s)
#> → Genotype these 10: 98% of the others then have a genotyped relative at  $\geq 0.25$ 
  ↳ (grandparent, half sib or closer) and 62% one at  $\geq 0.5$  (parent, full sib)
```

```
#> Coverage    mean best relationship 0.443 · median 0.500 · 10% of targets below 0.250
#> Compared with coverage mean best ≥ 0.25 ≥ 0.5
#> Optimised    0.499    0.443    98%    62%
#> Most connected 0.327    0.252    44%    20%
#> Random (mean) 0.419    0.354    79%    33%
#> Random      the set beats 100% of 50 random sets of the same size
#> First       G1_10, G2_12, F4, G1_9, G1_26, G2_4, F1, G1_24, ...
table(generation = ped$gen[match(gs$selected$id, ped$id)])
#> generation
#> 0 1 2
#> 3 4 3
```

What to look at: the comparison table. After genotyping the chosen 10, 98% of the others have a genotyped relative at 0.25 or closer (grandparent, half sib), against 79% for random sets and only 44% for the usual “most connected” rule, which picks the individuals with the largest total relationship and so piles up in one part of the pedigree. The set leans towards founders and early generations. In practice, candidates restricts the choice to individuals with DNA left, and must_include adds those already genotyped.

16 Genotyping, imputation and plots under one budget: `tso_imputation_plan()`

Most candidates are genotyped with a cheap low-density panel and imputed from a high-density reference set, and money spent on genotyping is not spent on plots. `tso_imputation_plan()` tries every split of one budget on high-density genotypes you already have. For each split it imputes the low-density lines, designs the training set on the imputed data, and scores it by its **realised** reliability: GBLUP with the imputed relationships, measured against the true ones.

```
ip <- tso_imputation_plan(tp, budget = 15000, cost_hd = 40, cost_ld = 12, plot_cost =
  ↪ 25,
                           n_ref = c(30, 60, 120), ld_markers = c(50, 200, 500))
ip
#> TSOpt genotyping, imputation and phenotyping plan · 300 lines x 958 HD markers ·
  ↪ budget 15,000 (HD 40, LD 12, plot 25) · 10 plans (5.4 s)
#> plan                                genotype plots imput. r K fidel. expected
  ↪ realised
#> 120 HD + 180 LD (500 markers)         6,960    270    0.844    0.988    0.579
  ↪ 0.518
#> 60 HD + 240 LD (500 markers)          5,280    270    0.756    0.979    0.578
  ↪ 0.511
#> 30 HD + 270 LD (500 markers)          4,440    270    0.656    0.972    0.572
  ↪ 0.507
#> 120 HD + 180 LD (200 markers)         6,960    270    0.809    0.961    0.583
  ↪ 0.468
#> 60 HD + 240 LD (200 markers)          5,280    270    0.736    0.933    0.588
  ↪ 0.440
#> 30 HD + 270 LD (200 markers)          4,440    270    0.648    0.904    0.590
  ↪ 0.427
#> all high density                      12,000    120    1.000    1.000    0.399
  ↪ 0.399
#> 30 HD + 270 LD (50 markers)           4,440    270    0.514    0.702    0.682
  ↪ 0.295
#> 120 HD + 180 LD (50 markers)          6,960    270    0.573    0.756    0.519
  ↪ 0.272
```

```
#> 60 HD + 240 LD (50 markers)          5,280    270    0.550    0.724    0.606
  ↳ 0.263
#> Best: 120 HD + 180 LD (500 markers), 270 plots (reliability +0.120 vs genotyping
  ↳ everyone at high density)
#> realised = reliability of GBLUP with the imputed relationships, on the true ones;
  ↳ expected = what the design believes.
```

What to look at: the gap between expected (what the design believes on the imputed data) and realised. Very sparse panels make the relationships look smoother than they are, and the design is overconfident.

17 Omics profiles: transcriptomes, metabolomes and spectra

A profile measured on a line (gene expression, metabolite levels, a near-infrared spectrum) predicts part of its performance. `tso_omics_kernel()` turns a table of profiles (one row per sample, one column per feature) into a kernel of profile similarities: it log-transforms counts, pre-treats spectra (standard normal variate and a Savitzky-Golay derivative), adjusts for batches, averages replicate samples and standardises the features. Given genotypes, it also reports how **genetic** the profiles are: the heritability of every feature, and how well the omics relationships agree with the genomic ones. `tso_data()` then uses it next to the genomic kernel.

Profiles are often measured on some lines only. Those lines are not dropped: the omics relationships of the others are predicted from their genomic relationships (the single-step idea, with the genomic matrix in the role of the pedigree). Here 200 of the 300 example lines have a (simulated) profile:

```
om <- tso_simulate_omics(tp$geno, features = 150, seed = 1) # your file:
  ↳ tso_omics_kernel("profiles.csv")
ok <- tso_omics_kernel(om$profiles[1:200, ], geno = tp)
ok
#> TSOpt omics kernel · OMICS (other, linear) · 200 lines x 150 features
#> Type other (detected)
#> Kernel linear on 150 standardised features · 200 lines
#> Genetics on 200 genotyped lines: median feature heritability 0.41 · agreement of
  ↳ omics and genomic relationships r = 0.55
#> Feature heritability: quartiles 0.27 / 0.41 / 0.56
#> The profiles are largely genetic: genomics can stand in for lines without a
  ↳ profile.
#> Next: tso_data(geno, kernels = list("A", OMICS = om)) (lines without a profile
  ↳ are filled from genomics)
tp_o <- tso_data(ex("example_panel.vcf.gz"), kernels = ok, verbose = FALSE)
tail(tp_o$log, 2)
#> [1] "Omics OMICS: 200 lines profiled, 100 without a profile predicted from genomics
  ↳ (median feature heritability 0.41)"
#> [2] "Kernels A 70% + OMICS 30% (centre; the design averages over 3 share scenarios)
  ↳ of the genetic and omics variance; 300 lines"
plan_o <- tso_design(tp_o, n = 60, target = targets, verbose = FALSE)
```

Does profiling pay? `tso_omics_plan()` answers it on pilot lines that are profiled: it tries every split of one budget between profiles and plots, predicts the missing profiles, designs the plots and scores every plan under the true profiles.

```
ok_all <- tso_omics_kernel(om$profiles)
op <- tso_omics_plan(tp, ok_all, budget = 6000, profile_cost = 4, plot_cost = 40,
```

```

n_profile = c(75, 150, 225), strategy = c("coverage", "targets"))
op
#> TSOpt omics profiling plan • OMICS • 300 pilot lines • budget 6,000 (profile 4, plot
  ↳ 40) • omics share 30%, h2 0.5 • 8 plans (3.1 s)
#> plan
#> profile every line
#> no profiling (genomics only)
#> 225 profiled (coverage)
#> 75 profiled (targets)
#> 225 profiled (targets)
#> 75 profiled (coverage)
#> 150 profiled (targets)
#> 150 profiled (coverage)
#> Best: profile every line, 120 plots (reliability +0.003 vs no profiling)
#> realised = reliability under the pilot's true profiles; expected = what the design
  ↳ believes;
#> fidelity = correlation of predicted and true omics relationships of unprofiled
  ↳ lines.

```

What to look at: the realised column, and the row without profiles. In our benchmark (benchmark/OMICS.md) the omics kernel changed which lines were worth phenotyping very little, but it raised prediction accuracy by 0.03 to 0.05 when half the signal ran through the profiles *and every line was profiled*. Profiling part of the lines rarely paid, and profiling everyone paid only when a profile cost about a twentieth of a plot, as near-infrared spectra do. Estimate the omics share from your trials (tso_variance(kernels = ok)) rather than assuming it.

18 From the plan to the field and the database

tso_export_field() writes the plan, or its field layout, for the tools your field team uses: a Field Book file, a FieldHub entry list, or a Breedbase trial-upload sheet. tso_brapi_write_list() saves the lines as a germplasm list in a BrAPI database (BMS, Breedbase), and tso_brapi_write_units() creates the plots of a study. tso_plan_json() writes a versioned plan file with a manifest of the inputs (tso_verify() checks a plan against data), and tso_analysis_script() writes the matching analysis model for sommer, ASReml-R or rrBLUP.

```

plan <- tso_design(tp, n = 40, target = targets, verbose = FALSE)
head(tso_export_field(plan, format = "fieldbook", trial = "YT2027"), 3)
#> plot_id row column plot block rep accession_name entry_type
#> 1 YT2027_1 NA NA 1 1 1 L234 test
#> 2 YT2027_2 NA NA 2 1 1 L226 test
#> 3 YT2027_3 NA NA 3 1 1 L235 test
tso_verify(plan, tp)
#> item recorded
#> 1 genotypes 3fa86357106aa9aa99ecea55c6f4fb91
#> 2 relationships 8e31335d468c084fa46b0981f5c28b9a
#> 3 units d96254fb7ae40a7c59fe47b13dba8f8c
#> current match
#> 1 3fa86357106aa9aa99ecea55c6f4fb91 TRUE
#> 2 8e31335d468c084fa46b0981f5c28b9a TRUE
#> 3 d96254fb7ae40a7c59fe47b13dba8f8c TRUE

```

19 Training in one programme, predicting another: `tso_transfer()`

Breeding networks share data. Can the trials of programme A train the predictions for programme B? `tso_transfer()` builds three designs of the same size and scores each on the lines of B: all n lines from A (*from only*); lines from A plus at most `n_local` “bridge” lines phenotyped in B (*with bridge*); and all n lines from B (the *in-programme* reference, what B could do alone). Transferability is the accuracy of the first relative to the third. Here groups G1 to G3 play programme A and G4 programme B:

```
trf <- tso_transfer(tp, from = c("G1", "G2", "G3"), to = "G4", n = 40,
                  n_local = 10)
trf
#> TSOpt transfer · train in G1+G2+G3 (225 lines) → predict G4 (75 lines) · 40 plots ·
  ↳ averaged over  $h^2 = 0.2, 0.5, 0.8$ 
#> → Training in G1+G2+G3 predicts G4 with 30% of the accuracy of training in G4;
  ↳ adding 10 local lines raises it to 71%
#>      design      from G1+G2+G3 from G4 CDmean  accuracy  relative
#>      from only      40          0      0.027   0.164    30%
#>      with bridge    30         10      0.150   0.387    71%
#>      in-programme   0          40      0.301   0.549   100%
#> Bridge    0 local: 30% · 2 local: 45% · 4 local: 53% · 6 local: 60% · 8 local: 66% ·
  ↳ 10 local: 71%
#> Related    G4 lines: mean relationship to G1+G2+G3 -0.062 · closest relative in
  ↳ G1+G2+G3 0.028 on average (0.574 within G4) · 0% have a relative  $\geq 0.2$  in G1+G2+G3
```

What to look at: training in G1 to G3 alone reaches only 30% of the accuracy of training in G4, because 0% of the G4 lines have a relative of 0.2 or more in the other groups. Adding 10 local lines raises it to 71%, and the Bridge line shows how fast: the first few local lines are worth the most (cost_local prices a local plot). The closer the programmes (the Related line), the more plots shared data saves.

20 What is not here

Major genes, haplotypes and other fixed effects (fixed =): “TSOpt guide 4”. Checking a plan, the value of each plot, how sure its advantage is, how many plots pay off, and the next stage and season: “TSOpt guide 5: Check, decide and plan ahead”.